

agent: discovery-technical-debt-agent cli: Claude Code CLI llm: claude-opus-4-8
run_id: 20260807T165658_j4iyle generated_at: 2026-08-07T12:36:37.834Z

8. Technical Debt Analysis

Objective: Establish the prerequisites for an Agentic Harness & Marketplace initiative — code repository health, third-party tool usage, AI tool usage, database usage, and development environment readiness.

Date: 2026-08-07 12:38:05 UTC | **Scope:** shende-shweta/pingcrm (branch master) — Laravel 11 / PHP 8.2 backend, Inertia.js, React 19 + TypeScript + Vite 7 frontend; PHPUnit + Vitest; SQLite default DB. ~78k PHP LOC / ~108k JS-TS LOC across an intentionally-generated legacy IVR surface.

Executive Summary

EXECUTIVE SUMMARY

PingCRM is a Laravel 11 + Inertia + React 19 demo CRM that has been deliberately extended with a large, machine-generated "legacy Enterprise IVR" surface (83 fat controllers, 12 legacy models, 47 identically-shaped `ivr_*` tables) documented in `DISCOVERY.md` as intentional technical debt for modernization workshops. On the positive side the repo has genuine hygiene fundamentals: a working `.gitignore`, both lock files committed, three GitHub Actions workflows (tests, static analysis, coding standards), an `.env.example`, and configured ESLint/Prettier/PHPStan tooling. The three most severe gaps are: (1) the database has **no foreign-key constraints anywhere** and a 47-table shared flat schema, pushing all integrity into application code and blocking safe service extraction; (2) **hard-coded production-style secrets are committed** in `config/ivr_legacy.php` (`master_api_key`, Salesforce `client_secret`, a plaintext password), so no clean automation checkout is possible; and (3) **near-zero test coverage** (4 PHPUnit files + 1 Vitest smoke test against ~186k LOC) means the CI gate cannot actually verify agent-authored changes. On agentic-harness readiness the verdict is **not ready today**: the codebase is highly enumerable and isolated (an ideal fan-out target), but the missing verification gate, absent container path, and committed secrets must be remediated before an agent fleet can be trusted to land changes.

Yes

TOP-LEVEL .GITIGNORE PRESENT

3

CI/CD WORKFLOWS FOUND

6 / 4

THIRD-PARTY PACKAGES DECLARED / WIRED

Yes

.ENV.EXAMPLE PRESENT

OVERALL CODEBASE RATING — TECHNICAL DEBT & AGENTIC READINESS

High Risk

Driven by D4 (no FK constraints + 47-table shared flat schema) and D6 (hard-coded secrets committed in config), compounded by a CI gate with near-zero test coverage.

Readiness Benchmark Ratings

One row per readiness dimension. "Measured" is the real state found; "Rating" is the band it falls into (worst-wins). This table is the source for the Overall Codebase Rating banner above.

#	Dimension	GOOD	MODERATE	HIGH RISK	Measured	Rating
D1	Code Repository Health	all checks pass	1–2 gaps	3+ gaps / no CI	<code>.gitignore</code> , 3 CI workflows, both lock files present; no CODEOWNERS/PR template, PHPStan level 1	MODERATE
D2	Third-Party Tool Usage	mostly wired & current	some unused/unwired	many unused/unmaintained	Sanctum/Glide/PHPStan wired; Guzzle indirect-only; <code>laravel/sail</code> declared but no container; <code>react-router-dom@5.2.0</code> stale duplicate of Inertia routing	MODERATE
D3	AI Tool / Agentic Readiness	ready	partial	not ready	Highly enumerable/isolated units (83 controllers, 47 tables) but no AI config and no verification gate (near-zero tests)	MODERATE
D4	Database Usage	sound	some gaps	no constraints / shared flat schema	Zero FK constraints; 47 identical flat <code>ivr_*</code> tables; non-idempotent seeder	HIGH RISK
D5	Development Environment	reproducible	partial	manual / fragile	<code>.env.example</code> + lint/format configured & CI-run; but no Dockerfile/compose/devcontainer and no local pre-commit hook	MODERATE
D6	Repo hygiene for automation — committed secrets (additional)	no secrets in tree	isolated demo creds	production-style secrets committed	<code>config/ivr_legacy.php</code> commits <code>master_api_key</code> , SF <code>client_secret</code> , plaintext password	HIGH RISK
D7	Observability / logging baseline (additional)	structured logs + monitoring	basic logging only	none	<code>LOG_CHANNEL=stack</code> → single flat file; no structured/JSON logging or monitoring config	MODERATE

Additional readiness gaps beyond the standard five dimensions **were** observed and are captured as **D6** (committed secrets) and **D7** (no observability baseline) above.

8.1 Code Repository

Check	Finding	Consequence
-------	---------	-------------

<code>.gitignore</code> coverage	Present (<code>.gitignore:1-20</code>). Covers <code>/vendor</code> , <code>/node_modules</code> , <code>.env</code> , <code>.env.backup</code> , <code>/public/build</code> , <code>/public/hot</code> , <code>/storage/*.key</code> , IDE dirs. A nested <code>database/.gitignore</code> ignores <code>*.sqlite*</code> .	Solid — the local SQLite DB and secrets file are not committable, so a clean checkout stays clean of local state.
CI/CD presence	3 workflows: <code>.github/workflows/tests.yml:1-80</code> (PHPUnit on push→master, all PRs, nightly cron, against MySQL 8), <code>.github/workflows/static-analysis.yml:1-15</code> (Larastan, reusable Laravel workflow), <code>.github/workflows/coding-standards.yml:1-8</code> (Pint auto-fix, <code>on: [push]</code>).	Tests do run on every PR, which is the single biggest strength here. But <code>coding-standards</code> <i>auto-fixes and pushes</i> rather than gating, and static analysis runs at PHPStan level 1 (<code>phpstan.neon:10</code>) — the lowest meaningful level, so almost nothing is actually caught.
Branch protection signals	No <code>CODEOWNERS</code> , no <code>.github/pull_request_template.md</code> , no required-checks config visible in the tree. Only <code>.github/FUNDING.yml</code> present.	Nothing in-repo forces the passing CI to be a merge gate; unreviewed/red-CI code can land on <code>master</code> . (Best-effort — server-side branch protection may exist but is not visible locally.)
Lock files committed	Both present: <code>composer.lock</code> (379 KB) and <code>package-lock.json</code> (297 KB).	Reproducible installs for both PHP and Node toolchains — good.

8.2 Third-Party Tools Usage

Security- / infrastructure-relevant declared dependencies and whether they are actually wired into code (not merely declared):

Package	Declared	Actually Wired?	Debt
<code>laravel/sanctum</code> (auth)	Y (<code>composer.json:18</code>)	Y — <code>HasApiTokens</code> trait on <code>app/Models/User.php:15</code> , <code>config/sanctum.php</code> , <code>personal_access_tokens</code> migration	Wired, but the generate (<code>routes/generated</code>) is unversioned and per auth/rate-limits — Sanc only.
<code>league/glide-symfony</code> (image serving)	Y (<code>composer.json:20</code>)	Y — used in <code>app/Http/Controllers/ImagesController.php</code>	Wired and used for on-
<code>larastan/larastan</code> + <code>phpunit</code> (quality)	Y (<code>composer.json:23,27</code>)	Y — <code>phpstan.neon</code> , <code>phpunit.xml</code> , CI workflows	Wired but under-utiliser PHPUnit suite has 4 fil
<code>guzzlehttp/guzzle</code> (HTTP client)	Y (<code>composer.json:15</code>)	N — no <code>GuzzleHttp\</code> / <code>Http::</code> reference found in <code>app/</code>	Present only as a trans dependency; no first-p exists. Declared-but-un
<code>laravel/sail</code> (Docker dev)	Y (<code>composer.json:24</code>)	N — no <code>docker-compose.yml</code> , <code>Dockerfile</code> , or <code>.devcontainer</code> in the tree	Sail is declared but the were never committed, work — a dead dev-en
<code>react-router-dom@5.2.0</code> (FE routing)	Y (<code>package.json:19</code>)	Partial — Inertia already owns routing via <code>@inertiajs/react@2.0.0</code>	A stale (v5, 2020-era) r alongside Inertia's own duplicated/redundant r outdated pin.

8.3 AI Tool Usage & Agentic Readiness

Existing AI tooling: None found. No `.cursor/` , `.kiro/` , `.github/copilot*` , `.cursorrules` , or `CLAUDE.md` exist inside the repository (the `.cursor/` dir in the CLI workspace is orchestration infrastructure, not part of `shende-shweta/pingcrm`). Maturity of AI-assisted tooling in the repo today: **zero**.

Code generators / scaffolding: The repo is substantially machine-generated. `tools/generate-legacy-enterprise-ivr.php` , `tools/generate-legacy-enterprise-ivr.mjs` , `tools/generate-legacy-enterprise-ivr-pass2.mjs` , and `tools/sync-ivr-legacy-routes.php` (documented in `DISCOVERY.md:12-18`) emit the entire legacy IVR surface. This means the codebase has an unusually **enumerable and structurally uniform** shape:

- **83** fat controllers under `app/Http/Controllers/Ivr/`
- **12** legacy models under `app/Models/Ivr/` and **17** god-service files under `app/Legacy/`
- **47** near-identical `ivr_*` module tables (all `tenant_id / name / json payload`), enumerated as a literal list in `database/migrations/2026_07_28_000001_create_ivr_legacy_tables.php`
- **83** React pages under `resources/js/Pages/Ivr/`

Honest assessment: The *decomposition* half of agentic readiness is excellent — an agent fleet could trivially enumerate "one task per IVR module / controller" and each unit is isolated enough to refactor independently. The *verification* half is missing: with only 4 PHPUnit files (`tests/Feature/ContactsTest.php` , `OrganizationsTest.php` , `Unit/ExampleTest.php` , plus `TestCase.php`) and a single `resources/js/test/smoke.test.ts` , the CI gate cannot confirm an agent-authored change didn't break behaviour. Additionally, 92 files use `extract()` and 105 use raw `DB::` query patterns — unsafe idioms that an automated refactor must handle carefully. Net: **partial** — ready to *fan out*, not yet ready to *trust the merge*.

92 files affected.

File	Line(s)	Issue	Action
<code>app/Http/Controllers/Ivr/AgentDeskDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses <code>extract()</code> — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace <code>extract()</code> with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/AgentDeskExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses <code>extract()</code> — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace <code>extract()</code> with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/AgentDeskImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses <code>extract()</code> — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace <code>extract()</code> with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/AgentDeskIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses <code>extract()</code> — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace <code>extract()</code> with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/AgentDeskStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses <code>extract()</code> — variable-injection idiom that obscures data flow and blocks safe	Replace <code>extract()</code> with explicit named variables so an agent can reason

		automated refactor / verification	about and verify each unit
<code>app/Http/Controllers/Ivr/AgentDeskSyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable- injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/AgentDeskUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable- injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/BusinessHoursDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable- injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/BusinessHoursExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable- injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/BusinessHoursImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable- injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/BusinessHoursIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable- injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/BusinessHoursStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable- injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/BusinessHoursSyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable- injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/BusinessHoursUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable- injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallAnalyticsDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable- injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallAnalyticsExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable- injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallAnalyticsImportController.php</code>	48, 61, 74, 87, 100, 113, 126,	Uses extract() — variable- injection idiom that obscures data flow and blocks safe	Replace extract() with explicit named variables so an agent can reason

	139, 152, 165, 178, 191 +43	automated refactor / verification	about and verify each unit
<code>app/Http/Controllers/Ivr/CallAnalyticsIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallAnalyticsStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallAnalyticsSyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallAnalyticsUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallFlowDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallFlowExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallFlowImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallFlowIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallFlowStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallFlowSyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallFlowUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallRecordingDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126,	Uses extract() — variable-injection idiom that obscures data flow and blocks safe	Replace extract() with explicit named variables so an agent can reason

	139, 152, 165, 178, 191 +43	automated refactor / verification	about and verify each unit
<code>app/Http/Controllers/Ivr/CallRecordingExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallRecordingImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallRecordingIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallRecordingStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallRecordingSyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallRecordingUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallRoutingDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallRoutingExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallRoutingImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallRoutingIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallRoutingStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CallRoutingSyncController.php</code>	48, 61, 74, 87, 100, 113, 126,	Uses extract() — variable-injection idiom that obscures data flow and blocks safe	Replace extract() with explicit named variables so an agent can reason

	139, 152, 165, 178, 191 +43	automated refactor / verification	about and verify each unit
<code>app/Http/Controllers/Ivr/CallRoutingUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CustomerProfileIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CustomerProfileStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/CustomerProfileUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/DidInventoryDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/DidInventoryExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/DidInventoryImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/DidInventoryIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/DidInventoryStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/DidInventorySyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/DidInventoryUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/HistoricalReportsDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126,	Uses extract() — variable-injection idiom that obscures data flow and blocks safe	Replace extract() with explicit named variables so an agent can reason

	139, 152, 165, 178, 191 +43	automated refactor / verification	about and verify each unit
<code>app/Http/Controllers/Ivr/HistoricalReportsExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/HistoricalReportsImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/HistoricalReportsIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/HistoricalReportsStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/HistoricalReportsSyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/HistoricalReportsUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/LiveMonitoringDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/LiveMonitoringExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/LiveMonitoringImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/LiveMonitoringIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/LiveMonitoringStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/LiveMonitoringSyncController.php</code>	48, 61, 74, 87, 100, 113, 126,	Uses extract() — variable-injection idiom that obscures data flow and blocks safe	Replace extract() with explicit named variables so an agent can reason

	139, 152, 165, 178, 191 +43	automated refactor / verification	about and verify each unit
<code>app/Http/Controllers/Ivr/LiveMonitoringUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/PromptLibraryDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/PromptLibraryExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/PromptLibraryImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/PromptLibraryIndexController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/PromptLibraryStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/PromptLibrarySyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/PromptLibraryUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/QueueManagementDestroyController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/QueueManagementExportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/QueueManagementImportController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/QueueManagementIndexController.php</code>	48, 61, 74, 87, 100, 113, 126,	Uses extract() — variable-injection idiom that obscures data flow and blocks safe	Replace extract() with explicit named variables so an agent can reason

	139, 152, 165, 178, 191 +43	automated refactor / verification	about and verify each unit
<code>app/Http/Controllers/Ivr/QueueManagementStoreController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/QueueManagementSyncController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Http/Controllers/Ivr/QueueManagementUpdateController.php</code>	48, 61, 74, 87, 100, 113, 126, 139, 152, 165, 178, 191 +43	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Legacy/Services/AgentDeskGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Legacy/Services/BusinessHoursGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Legacy/Services/CallAnalyticsGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Legacy/Services/CallFlowGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Legacy/Services/CallRecordingGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Legacy/Services/CallRoutingGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Legacy/Services/CustomerProfileGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Legacy/Services/DidInventoryGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Legacy/Services/HistoricalReportsGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71,	Uses extract() — variable-injection idiom that obscures data flow and blocks safe	Replace extract() with explicit named variables so an agent can reason

	79, 87, 95, 103 +33	automated refactor / verification	about and verify each unit
<code>app/Legacy/Services/LiveMonitoringGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Legacy/Services/PromptLibraryGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit
<code>app/Legacy/Services/QueueManagementGodService.php</code>	15, 23, 31, 39, 47, 55, 63, 71, 79, 87, 95, 103 +33	Uses extract() — variable-injection idiom that obscures data flow and blocks safe automated refactor / verification	Replace extract() with explicit named variables so an agent can reason about and verify each unit

8.4 Database Usage

Check	Finding	Consequence
Schema design	No foreign-key constraints anywhere. Relationships are plain <code>integer('account_id')->index()</code> / <code>unsignedInteger('organization_id')->index()</code> columns (<code>create_contacts_table.php:16-17</code> , <code>add_account_id_to_ivr_tables.php</code>). Indexes exist beyond the PK, but referential integrity is absent.	Every integrity rule (valid <code>account_id</code> , <code>organization_id</code> , tenant scoping) lives in PHP; a bad write or a direct DB edit silently orphans rows, and the app cannot rely on the DB to reject them.
Migration hygiene	Good defensive style — legacy migrations guard every step with <code>Schema::hasTable()</code> / <code>Schema::hasColumn()</code> (<code>create_ivr_legacy_tables.php:27-29</code> , <code>add_account_id_to_ivr_tables.php:32-70</code>) and provide symmetric <code>down()</code> methods. The <code>account_id</code> backfill runs an idempotent <code>whereNull(...)->update(...)</code> .	Migrations are re-runnable and reversible — the one genuinely healthy part of the data layer.
Data ownership	Shared, flat schema. 47 <code>ivr_*</code> module tables share one identical shape (<code>tenant_id</code> , <code>name</code> , <code>json payload</code>) rather than being modelled per domain; CRM tables (<code>accounts</code> , <code>organizations</code> , <code>contacts</code> , <code>users</code>) are separate but IVR data is a single flat pool retro-fitted with a nullable <code>account_id</code> .	Blocks safe extraction into separate services later — there is no schema boundary per domain, so a "queue management" service and a "billing meter" service would today read the same undifferentiated table pattern.
Seed / sample data hygiene	<code>database/seeder/DatabaseSeeder.php:19-42</code> is not idempotent — it calls <code>Account::create(...)</code> and <code>User::factory()->create(...)</code> unconditionally (no <code>firstOrCreate</code> / <code>truncate</code>), so a second <code>db:seed</code> duplicates Acme Corporation and all demo data. It also seeds a hard-coded password <code>'secret'</code> . Scheduled <code>migrate:fresh --seed</code> reset commands exist (git history) as a workaround.	Re-seeding on a shared/CI environment silently doubles data; the workaround (full <code>migrate:fresh</code>) is destructive and unsuitable for anything but a throwaway DB.

8.5 Development Environment

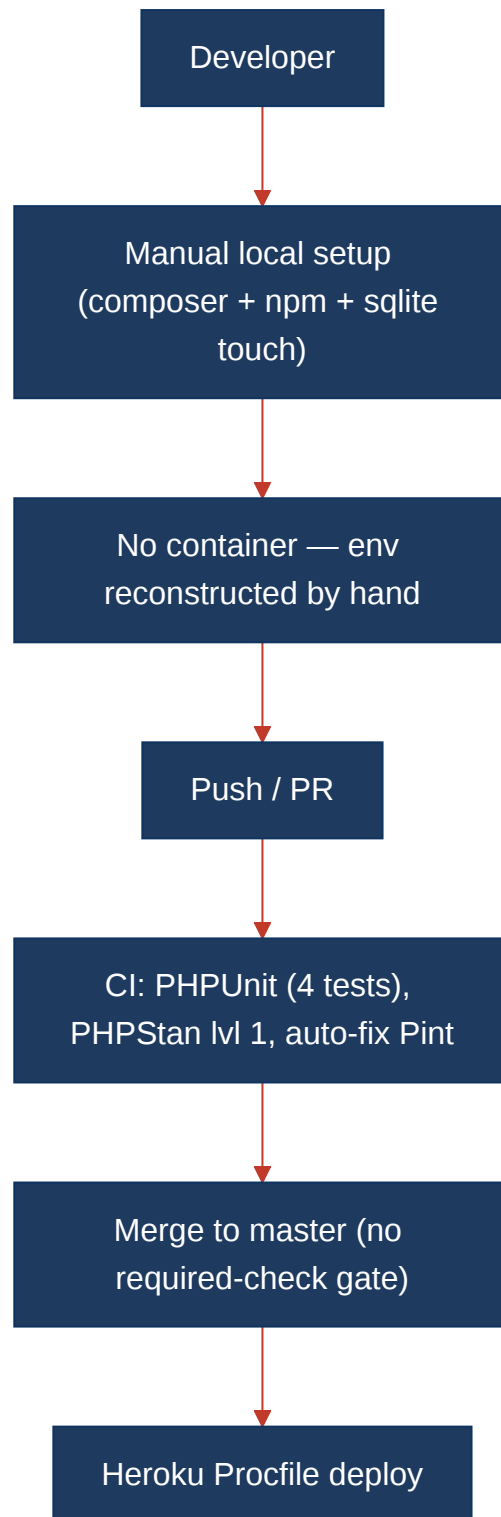
Check	Finding	Consequence
<code>.env.example</code>	Present (<code>.env.example:1-68</code>) and consistent with README setup (<code>DB_CONNECTION=sqlite</code> , <code>APP_KEY=</code> to be generated, mail/redis/pusher placeholders).	Onboarding path is documented and reproducible for the core CRM.
OS portability	Setup is generic (<code>php artisan serve</code> , <code>npm run dev</code> , <code>touch database/database.sqlite</code>) with no OS-specific tooling in the README. <code>Procfile</code> targets Heroku (<code>heroku-php-apache2</code>).	Reasonably portable across macOS/Linux/WSL; no single-vendor lock-in blocking contributors.
Containerization	None committed. No <code>Dockerfile</code> , <code>docker-compose.yml</code> , or <code>.devcontainer/</code> in the tree, even though <code>laravel/sail</code> is a declared dev dependency (<code>composer.json:24</code>).	No reproducible container path — every contributor and the CI runner reconstruct PHP/Node/DB versions by hand, inviting environment drift between "works on my machine" and CI.
Code style enforcement	Configured and partially enforced: ESLint (<code>.eslintrc.cjs</code>) + Prettier (<code>.prettierrc</code>) + <code>fix-code-style</code> npm script, PHPStan (<code>phpstan.neon</code>), plus the <code>coding-standards</code> CI workflow. But there is no <code>.husky/</code> /pre-commit hook, and the CI standards job <i>auto-fixes and pushes</i> rather than failing the build.	Style tooling exists, yet nothing blocks a non-conforming commit locally, and the CI job masks violations by rewriting them — so style/lint drift is not actually prevented, only cosmetically corrected after the fact.

8.6 Prerequisites for Agentic Harness & Marketplace Readiness

Prerequisite	Current State	Gap
Enumerable work queue (clear, listable units of refactor work)	Strong — 83 IVR controllers, 47 uniform tables, 83 React pages, all machine-enumerable from <code>DISCOVERY.md</code> and the generator scripts	Not yet expressed as an explicit backlog/queue; the enumeration lives implicitly in generator arrays, not a task manifest an agent can consume
Isolated, verifiable units of work	Partial — units are structurally isolated, but "verifiable" fails: 4 PHPUnit + 1 Vitest test cover ~186k LOC	No characterization/regression tests around the legacy surface, so a refactor's correctness cannot be asserted
CI gate to accept agent-authored output	Partial — CI runs tests on every PR, but with near-zero coverage the gate is a formality; PHPStan is level 1; standards job auto-fixes instead of gating	Raise coverage + PHPStan level and make checks <i>required</i> so a green CI actually means "safe to merge"
Repo hygiene for automation (clean checkout, no secrets)	Fails — <code>config/ivr_legacy.php</code> commits real-looking secrets; 92 <code>extract()</code> + 105 raw- <code>DB::</code> sites	Move secrets to env, add a secret-scan step, and neutralize unsafe idioms so an agent runs on a clean, safe tree
Marketplace packaging readiness	Weak — no container image, no <code>.devcontainer</code> , <code>laravel/sail</code> declared without compose files	Add a Dockerfile/devcontainer so an agent task can spin up an identical, disposable environment

8.7 Diagrams

Current dev / delivery flow



Agentic harness readiness target



Improvement roadmap



8.8 Actions Required

Only dimensions/gaps rated Moderate or High Risk are listed; each has a concrete action.

Gap	Action	Rating	Priority
Hard-coded secrets committed in <code>config/ivr_legacy.php</code> (<code>master_api_key</code> , SF <code>client_secret</code> , plaintext password)	Move all values to <code>env()</code> reads + <code>.env.example</code> placeholders, rotate the exposed credentials, and add a secret-scanning step (e.g. gitleaks) to CI	HIGH RISK	CRITICAL
No foreign-key constraints anywhere; integrity lives only in PHP	Add <code>foreignId(...)->constrained()</code> (or explicit <code>foreign()->references()</code>) for <code>account_id</code> / <code>organization_id</code> / <code>tenant_id</code> in a new migration; verify no orphaned rows first	HIGH RISK	HIGH
47 <code>ivr_*</code> tables share one flat schema — blocks per-domain service extraction	Define per-domain schemas/ownership for the highest-value modules before any extraction; treat the flat <code>json payload</code> tables as a migration target, not an end state	HIGH RISK	MEDIUM
Near-zero test coverage → CI gate cannot verify agent output	Add characterization tests around CRM + top IVR modules; wire a coverage threshold and raise PHPStan from level 1 toward 5+	MODERATE	HIGH
No merge gate (no CODEOWNERS/PR template/required checks)	Add <code>CODEOWNERS</code> + a PR template and mark <code>tests</code> / <code>static analysis</code> as required status checks on <code>master</code>	MODERATE	MEDIUM
No container path; <code>laravel/sail</code> declared without compose files	Commit a <code>docker-compose.yml</code> / <code>Dockerfile</code> (or <code>.devcontainer/</code>) so contributors, CI, and agent tasks share one disposable environment	MODERATE	MEDIUM
Style tooling not enforced locally; CI standards job auto-pushes fixes instead of gating	Add a pre-commit hook (husky + lint-staged / Pint) and change the CI job to <i>fail</i> on violations rather than rewrite them	MODERATE	LOW
Non-idempotent <code>DatabaseSeeder</code> duplicates data on re-run	Make seeders idempotent (<code>firstOrCreate</code> / <code>updateOrCreate</code> or guarded truncation) so seeding is safe on shared/CI databases	HIGH RISK	MEDIUM
Declared-but-unwired deps: <code>guzzle</code> (unused), <code>react-router-dom@5.2.0</code> (stale, duplicates Inertia)	Remove <code>react-router-dom</code> (Inertia owns routing) and drop/justify <code>guzzle</code> ; prune to reduce install and audit surface	MODERATE	LOW
No observability/logging baseline (<code>LOG_CHANNEL=stack</code> → single flat file)	Add structured/JSON logging and a minimal monitoring hook so agent-driven changes are observable in a running environment	MODERATE	LOW

92 <code>extract()</code> + 105 raw <code>DB::</code> sites obstruct safe automated refactor	Systematically replace <code>extract()</code> with explicit variables and parameterize raw queries (see §8.4 affected-files)	MODERATE	MEDIUM
--	---	----------	--------

8.9 Expected Outcomes

- **A clean, safe checkout:** no secrets in the tree, a secret-scan gate, and neutralized `extract()` /raw-SQL idioms — the precondition for any agent to run on the repo.
- **A CI gate that means something:** meaningful test coverage plus a raised PHPStan level and required status checks, so a green build genuinely authorizes an agent-authored merge.
- **A durable data layer:** enforced foreign keys, per-domain table ownership, and idempotent seeders — removing the largest modernization and extraction blocker.
- **Reproducible environments everywhere:** a committed container/devcontainer path aligning contributor, CI, and agent-task environments, ending drift.
- **A foundation for the agentic harness & marketplace:** the repo's already-enumerable IVR surface expressed as an explicit work queue, feeding isolated, CI-verified, human-reviewed agent tasks.